

Self-evolving Agentic Customer Support System at LinkedIn

Chih Hui Wang*, Mengdie Tu*, Qianyun Zhang*, Wei Wu*,
Lili Zhou, Mingqi Shen, Changshuai Wei†

LinkedIn

{ciwang, metu, qazhang, wwul, lizhou, minshen, chawei}@linkedin.com

Abstract

Enterprise support agents operate in rapidly changing environments where policies, product capabilities, and knowledge bases evolve continuously, making static assistants brittle and costly to maintain. We present LinkedIn’s self-evolving agentic support system, which integrates retrieval-augmented generation with evolutionary auto-prompting and a modular, production-aligned evaluation framework to enable safe, continuous improvement without retraining foundation models. The system treats prompts, retrieval, and evaluation as a closed-loop, versioned workflow with operational guardrails. Offline simulations and ablations show clear quality gains over vanilla RAG and baseline agents, including reduced hallucinations and improved response completeness. In a two-week user-randomized A/B test on LinkedIn’s production support traffic, the integrated self-evolved workflow increased QA self-serve by 9.0 percentage points, cancellation self-serve by 4.8 points, and routing accuracy by 30.6 points. These results demonstrate a practical path to scalable, self-evolving AI agents in real-world enterprise settings.

1 Introduction

Enterprise customer support is increasingly mediated by AI agents, yet real-world deployments face a reliability problem qualitatively different from standard chatbot benchmarks. At LinkedIn, support spans many product surfaces, multiple lines of business, and dozens of languages, while the underlying ecosystem changes continuously: product launches reshape the knowledge base, content is published or deprecated, retrieval indices are refreshed, prompts evolve, and underlying LLMs and tool APIs change over time. Even small upstream changes can induce regressions—hallucinations, stale guidance, broken tool flows, or inconsistent

multilingual tone—making a “set-and-forget” assistant brittle and expensive to maintain. Traditional production support agents—built on a handcrafted prompt, a fixed retrieval pipeline, and periodic human QA—do not scale with this velocity, and lack a principled feedback loop that can detect regressions, localize failures to specific components, and adapt without heavy human intervention.

Three lines of recent work motivate the design we present. *Prompt optimization* treats prompts as programs, with score-based search (APE (Zhou et al., 2023)), LLM-as-optimizer (OPRO (Yang et al., 2024)), evolutionary search (PromptBreeder, EvoPrompt (Fernando et al., 2023; Guo et al., 2024)), and declarative compilation (DSPy (Khat-tab et al., 2023)) all showing that prompts can be evolved offline against task metrics and shipped as versioned artifacts. *LLM-as-a-judge* evaluation enables scalable assessment of multi-dimensional quality (Zheng et al., 2023; Liu et al., 2023), with documented biases that can be mitigated through structured rubrics, length control (Dubois et al., 2024), and open evaluator models (Kim et al., 2024). *Retrieval-augmented generation* grounds responses in proprietary knowledge (Lewis et al., 2020b; Karpukhin et al., 2020; Gao et al., 2023a), and recent self-reflective variants (Self-RAG, CRAG (Asai et al., 2024; Yan et al., 2024)) treat retrieval as an explicit agent action rather than a preprocessing step, aligning with reasoning-and-acting paradigms (Yao et al., 2023).

For enterprise support, the operational implication of these advances is that prompts, retrievers, and evaluators all become *versioned artifacts* that can change frequently—and often function like policy changes affecting routing, escalation thresholds, safety posture, and brand tone across multiple products and languages. A self-evolving system therefore requires not just optimization, but the engineering discipline to deploy these changes safely: regression testing, staged rollout, gated promotion,

*Equal contribution

†Corresponding author

and rollback. Our evaluation layer must in turn be reliable enough to drive these decisions at scale, robust to knowledge drift, and decomposed into interpretable signals so failures can be attributed and acted on.

We co-evolve the three layers under strict version control, deliberately avoiding fine-tuning because support knowledge changes too quickly for re-training cycles and because retrieval-based knowledge injection often outperforms unsupervised fine-tuning for factual updates (Ovadia et al., 2024; Soudani et al., 2024). Our evaluation layer is structured as specialized evaluator agents (Wu et al., 2023), and our operational “memory” is implemented as bounded, auditable, versioned artifacts rather than unconstrained long-horizon memory (Packer et al., 2023; Shinn et al., 2023).

Contributions. We (1) describe an end-to-end self-evolving support agent that remains reliable under continual upstream change via closed-loop evaluation and optimization; (2) present an automatic prompt evolution engine suitable for enterprise constraints (tone, policy, multilingual); (3) propose a modular, multi-signal evaluation framework actionable for debugging at scale; and (4) detail a version-controlled RAG-and-tools layer supporting reproducibility, safe rollout, and rollback.

2 Methodology

2.1 Overall system

Figure 1 shows the Support AI Agent as a closed-loop, self-evolving system in which prompting, retrieval, and evaluation form an explicit feedback cycle. At inference time the agent is conditioned by a system prompt from the Automatic Prompt Engine that encodes task instructions, tone, and implicit policies for reasoning depth and tool use. RAG is exposed as an explicit action rather than a preprocessing step, letting the agent decide when to retrieve and how to integrate evidence, consistent with reasoning-and-acting paradigms (Yao et al., 2023; Schick et al., 2023). Outputs are scored by a modular evaluation framework that decomposes quality into grounding, intent alignment, multilingual fidelity, and stylistic compliance using rule-based diagnostics and LLM-based judges (Zheng et al., 2023); aggregate fitness signals feed back into the prompt engine, which refines the system prompt via evolutionary search. The architecture exposes two coupled loops—an inner inference

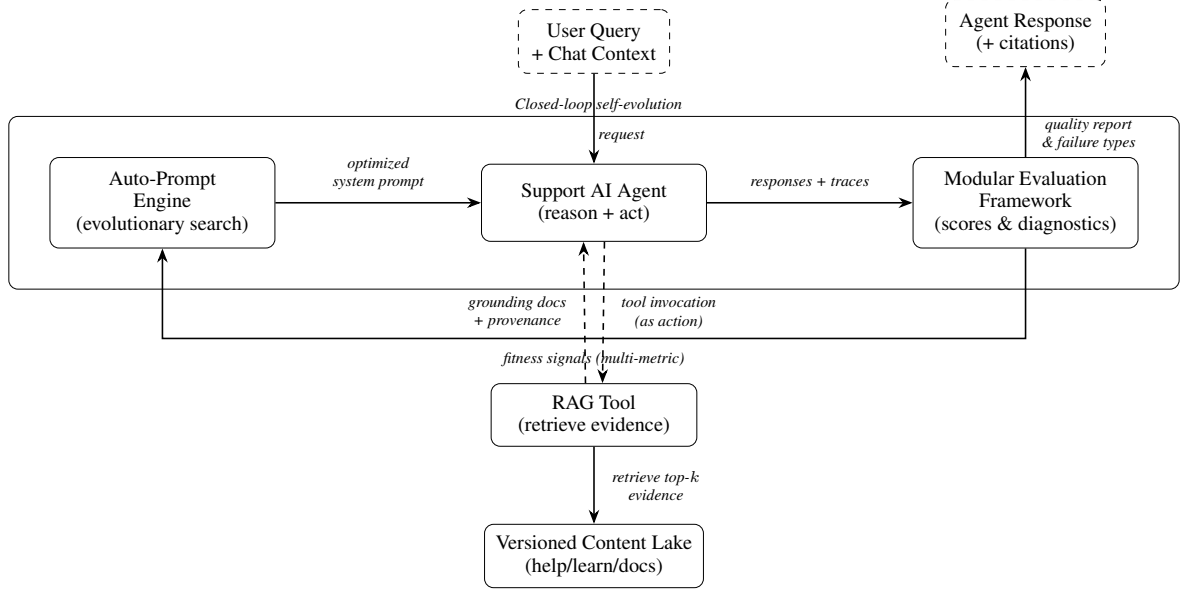
loop (agent $\leftrightarrow$ RAG $\leftrightarrow$ content lake) and an outer optimization loop (auto-prompt $\rightarrow$ agent $\rightarrow$ evaluator $\rightarrow$ auto-prompt)—enabling scalable self-improvement and rapid adaptation across products and languages without modifying the base model or retrieval corpus.

2.2 Auto-Prompt

Manual prompt development is a major bottleneck in production: in our deployment, a single line-of-business iteration can take weeks. We replace this with an Automatic Prompt Engineering Engine (“Auto-Prompt”) that formulates prompt optimization as evolutionary search inspired by genetic algorithms (Holland, 1992; Whitley, 1994). Domain-specific business rules guide an LLM-driven initialization stage, after which a population of candidate prompts evolves through selection, crossover, and mutation, evaluated each generation against task-specific metrics; only the strongest survive into the next iteration. The full pseudocode is given in Algorithm 1.

Evolutionary search fits this setting because prompt optimization is black-box, non-differentiable search over discrete text, with fitness supplied by LLM and rule-based evaluators. Unlike single-candidate hill climbing, a population preserves diverse combinations of tone, tool-use policy, reasoning depth, and response structure; crossover recombines complementary instructions discovered in different lineages, while mutation provides local exploration. APE and OPRO optimize prompts through score-guided generation (Zhou et al., 2023; Yang et al., 2024), and DSPy compiles declarative programs against metrics (Khattab et al., 2023); our use of constrained evolution emphasizes population diversity, recombination, and immutable enterprise-policy filters. The ablation in Table 2 tests whether both evolutionary operators are necessary.

To prevent evolutionary drift from violating safety or policy requirements, business rules are treated as immutable hard constraints rather than optimization objectives: violating prompts are filtered or assigned zero fitness prior to selection. Crossover and mutation therefore operate within a constrained prompt space that preserves required behaviors by construction, ensuring evolutionary gains target accuracy without eroding compliance or product invariants.



What evolves: prompt policies (tool use, tone, structure)
What stays fixed per iteration: base LLM¹ + content snapshot

Figure 1: Self-evolving Support AI Agent architecture.

2.3 Retrieval Augmented Generation (RAG)

Support knowledge is proprietary and changes constantly, so we ground responses in retrieved content rather than parametric model knowledge (Lewis et al., 2020b; Guu et al., 2020). We depart from fixed retrieval pipelines and expose RAG as a *tool* the agent invokes within its reasoning loop (Yao et al., 2023; Schick et al., 2023), letting the model decide when to retrieve and how to query, mirroring recent agentic RAG systems (Press et al., 2023; Shinn et al., 2023).

Our unified knowledge base consolidates three content sources: help articles, learning content, and product documentation microsites. They are refreshed by an ETL (Extract–Transform–Load) pipeline that chunks, embeds, facets, and retires content before loading it into the versioned content lake. Retirement is enforced through a snapshot pointer, so each refresh scopes retrieval to the latest snapshot and supersedes prior content without deleting it. Each document is tagged with three facets—*product* (line of business), *locale* (7+ languages), and *snapshot date*—enabling scoped, reproducible retrieval (Lin et al., 2021; Gao et al., 2023b). At query time, hybrid dense–sparse retrieval is followed by semantic re-ranking (Berntson, 2023; Nogueira and Cho, 2019), returning ranked grounding documents with prove-

nance (URL, locale, source) for citation and audit.³

2.4 Evaluation Framework

Operating across 36+ languages and many product lines with a constantly drifting knowledge base, we cannot rely on fixed gold sets or static benchmarks (Lewis et al., 2020a; Longpre et al., 2021). We instead use a modular, agent-aware framework that decomposes quality into interpretable dimensions—grounding, intent understanding, tool-execution correctness, and multilingual fidelity—each scored by specialized evaluators combining rule-based checks with LLM judgments. Multilingual quality is assessed by dedicated modules that detect untranslated spans, verify terminology, and measure fluency via XGLM perplexity (Lin et al., 2022), while an LLM evaluator scores semantic fidelity under realistic dialog conditions. Production monitoring spans all supported locales, while the controlled benchmark in Section 4.4 focuses on English↔Chinese as a challenging distant-language pair.

Unlike static benchmarks, evaluation is conditioned on the context retrieved at inference time, addressing knowledge drift directly (Zheng et al., 2023). Signals are aggregated through a consensus layer calibrated against periodic human annotations, stored in a versioned replayable format,

³GPT-4o-mini (via Azure OpenAI service) was used as the base LLM in production at the time of writing.

Algorithm 1 Genetic Prompt Optimization

Require: Business rules $\mathcal{R}$, evaluation dataset $\mathcal{D}$, population size N , elite size K , maximum generations G , scoring function $\mathcal{S}(\cdot)$

Ensure: Optimized prompt p^*

```
1: Initialize prompt population  $P_0 \leftarrow$   
   LLMGENERATE( $\mathcal{R}, N$ )  
2: Initialize evaluation cache  $\mathcal{C} \leftarrow \emptyset$   
3: for  $g = 1$  to  $G$  do  
4:   for each prompt  $p \in P_{g-1}$  do  
5:     if  $p \notin \mathcal{C}$  then  
6:       Run LLM inference on  $\mathcal{D}$  using  $p$   
7:       Compute fitness  $f(p) \leftarrow \mathcal{S}(p, \mathcal{D})$   
8:       Store  $(p, f(p))$  in cache  $\mathcal{C}$   
9:     else  
10:      Retrieve  $f(p)$  from  $\mathcal{C}$   
11:    end if  
12:  end for  
13:  Select top- $K$  elite prompts  $E_g \subset P_{g-1}$  by  
   fitness  
14:  Initialize new population  $P_g \leftarrow E_g$   
15:  while  $|P_g| < N$  do  
16:    Sample parents  $(p_i, p_j)$  from  $E_g$   
17:     $p' \leftarrow \text{SEMANTICBLEND}(p_i, p_j)^1$   
18:     $p'' \leftarrow \text{MUTATE}(p')^2$   
19:    Add  $p''$  to  $P_g$   
20:  end while  
21: end for  
22: return  $p^* \leftarrow \arg \max_{p \in P_G} f(p)$ 
```

and consumed by upstream optimization (auto-prompt and RAG iteration), closing the loop between agent behavior, measurement, and controlled self-evolution.

3 Engineering Architecture and Deployment

Continuous evolution of prompts, retrieval, and evaluation expands the failure surface relative to static deployments. The production architecture (Figure 2) is structured around four principles.

Reliability as a prerequisite for self-evolution.

A lightweight orchestration and fallback layer mediates user traffic with deterministic routing, error handling, and graceful degradation. This isolates end users from rapid iteration in the agent layer and enables fallback to human-assisted support when downstream AI components degrade or fail.

Runtime as a controlled mutation boundary.

Rather than embedding behavior in code, prompts are assembled dynamically, context-construction policies are swappable, and tool usage is declaratively configured. Prompt evolution, retrieval tuning, and workflow updates therefore ship through configuration and artifact updates rather than code deploys, allowing behavioral improvements to propagate without redeploying the runtime itself.

Memory as versioned artifacts, not latent state.

The runtime carries no cross-session latent state beyond short-horizon, task-scoped context required for conversational continuity. Longer-lived knowledge is promoted only through explicitly versioned and evaluated artifacts, enabling auditability, reproducibility, controlled rollback, and principled promotion of improvements while preventing uncontrolled memory drift.

Telemetry as an evaluation substrate. Execution telemetry serves both monitoring and automated evaluation. Structured event streams feed nearline quality checks and offline evaluators measuring groundedness, relevance, completeness, and policy compliance, forming the control plane that drives prompt evolution, retrieval tuning, and gated deployment without direct model retraining.

Optimization cadence. The outer optimization loop typically runs weekly and can also be triggered by content refreshes or detected regressions. One end-to-end evolution cycle takes hours to days depending on dataset size and fitness-evaluation budget. Candidate prompts and retrieval configurations are promoted only after offline regression checks and staged rollout; the execution plane can roll back to the prior versioned artifact if guardrails fail.

Deployment learnings. In production, three patterns proved load-bearing. *Centralized execution* through configuration, modular prompts, and declarative tool selection—rather than per-LoB code forks, or separately maintained code paths for individual lines of business—was critical to avoiding behavioral divergence and ensuring improvements generalized across workflows. *Layered fail-safes* such as targeted query redirection allowed traffic to route to a known-safe content pool when a polluted vector index required hours-long reindexing, cutting recovery from hours to minutes and limiting manual intervention. Finally,

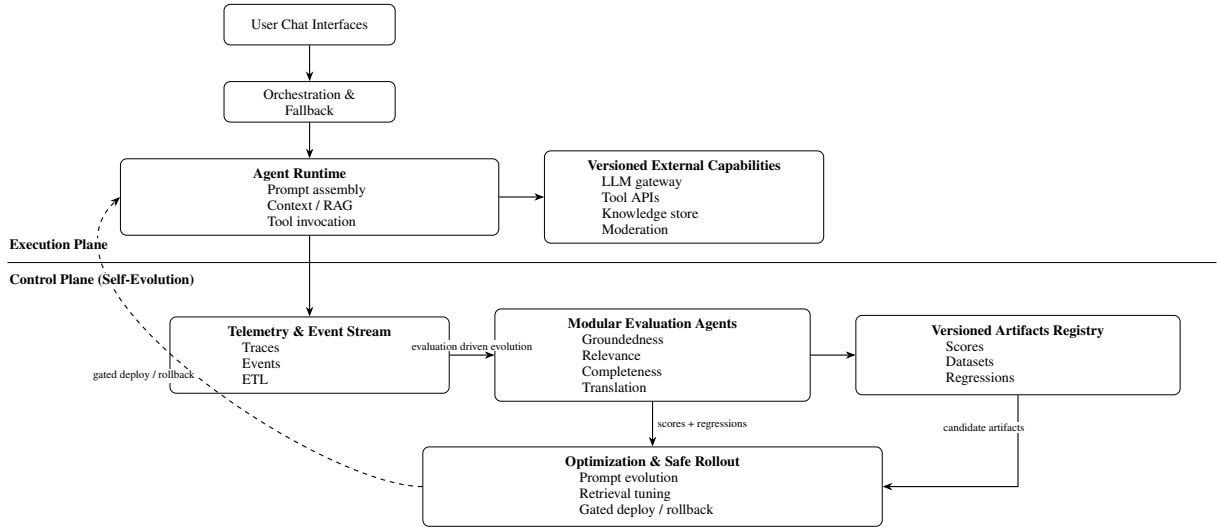


Figure 2: Production architecture for a self-evolving support agent. Optimization and Safe Rollout consumes evaluator scores and regression signals together with candidate versioned artifacts, then emits gated deployment or rollback decisions to the execution plane.

staged rollout of new execution patterns in green-field workflows—new, low-traffic workflows used as controlled proving grounds—before extension to mature traffic let evolution mechanisms and evaluation signals mature before broader deployment. Together these practices sustain continuous improvement under rapidly changing models, policies, and knowledge.

4 Simulation and Ablation Studies

We evaluate the agent through controlled simulations that replay anonymized historical support interactions and synthetic edge cases across multilingual, product-diverse, and evolving knowledge settings. Each simulation acts as a system-level ablation, comparing configurations with different agent capabilities enabled under identical evaluation criteria.

4.1 RAG Simulation

We compare four response-generation configurations using an LLM-based evaluator (GPT-4.1 via Azure OpenAI service): Vanilla RAG, RAG-only agents (ReAct-style and OpenAI tool invocation⁴), and the full Support AI Agent. The production generator is GPT-4o-mini while the offline judge is GPT-4.1, so they are different models. We measure overall quality, hallucination rate, and completeness with respect to retrieved support articles; a response is hallucinated if it contains any factual

claim not attributable to the retrieved content. Table 3 calibrates the judge against human labels.

As shown in Table 1, vanilla and agentic RAG configurations hallucinate at 4.8–6.2%, while the full Support AI Agent reaches <0.1% with the highest overall score (2.78) and completeness (87.8%). The gain comes from explicit agentic constraints that prioritize synthesis from retrieved authoritative content and suppress reliance on parametric knowledge when grounded sources exist.

4.2 Auto-Prompt Simulation

On the routing intent detection dataset, we iteratively optimize LLM-generated initial prompts across generations using the genetic algorithm pipeline (Algorithm 1). Table 2 shows that prompt accuracy improves consistently only when both crossover and mutation are enabled—increasing average accuracy from 62.6% to 68.0% after two generations, with the best prompt reaching 73.3%. Removing either operator slows improvement and saturates early, indicating that prompt *evolution*, not random sampling, is what drives sustained gains.

4.3 Evaluator-Signal Ablation

Beyond the system-level simulations above, we ablate individual evaluator signals to isolate their contribution, replaying each configuration on an anonymized validation set and comparing against a human-labeled reference. As shown in Table 3, response groundedness is the dominant driver of evaluator reliability—removing it causes the largest

⁴OpenAI tool agent is accessed via Azure OpenAI service

Table 1: RAG Simulation Results

Setting ¹	Overall Score ²	Hallucination ³	Completeness ⁴
Vanilla RAG	2.59	5.3%	78.7%
RAG Only (ReAct Agent)	2.56	4.8%	79.0%
RAG Only (OpenAI Tool Agent)	2.48	6.2%	75.0%
Support AI Agent	2.78	<0.1%	87.8%

¹ The simulation uses an anonymized, human-validated set of 100 support interactions with multiple inference runs per interaction.

² Overall score is on a 0–3 scale to measure the overall quality of the response.

³ A response is labeled as hallucinated if it contains any factual claim that cannot be attributed to the retrieved support articles provided to the model at inference time.

⁴ A response is labeled as complete if it fully addresses all aspects of the user inquiry.

Table 2: Automatic Prompt Optimization Simulation

Generation	Crossover + Mutation	No Mutation	No Crossover
0 (Baseline)	62.6 / 66.7	62.6 / 66.7	62.6 / 66.7
1	66.7 / 73.3	63.3 / 66.7	62.6 / 66.7
2	68.0 / 73.3	64.0 / 66.7	63.3 / 66.7

The simulation uses a curated, human-validated intent-detection dataset (Search Knowledge Base vs Direct to Consultant; $N = 30$). Metrics are average prompt accuracy / best prompt accuracy per generation.

Table 3: Ablation Study of RAG Auto-Evaluation Dimensions

Evaluation Setting	Alignment ¹ w/ Human Labels ²
Full RAG Evaluation (Baseline)	87%
A1: No Groundedness	76% (-11)
A2: No Content Relevance	82% (-5)
A3: No Completeness	86% (-1)
A4: Overall Score Only	67% (-20)

¹ Alignment is measured against human-labeled ground truth on a randomly sampled set of 100 premium-tier subscription support chats, with multiple runs per interaction.

² Three trained reviewers, blind to system condition, followed standardized guidelines. Raw agreement was 92%; disagreements were resolved by majority vote.

drop in alignment with human labels, while removing completeness barely changes it—and collapsing all signals into a single overall score performs worst, obscuring critical failure modes. These findings justify retaining groundedness, relevance, and completeness as first-class, independently reported dimensions (Qiao et al., 2025; Park et al., 2025; Es et al., 2023).

4.4 Multilingual Translation Evaluation

This evaluator-design ablation tests the modular multi-agent evaluator that produces the fitness sig-

Table 4: Translation Quality Evaluation Result¹

Evaluation Paradigm	Accuracy (%)
Model-based eval (COMET)	49.7
Pure LLM single-agent eval	76.5
Modular multi-agent eval	84.8

¹ Evaluation is performed on an anonymized support chat dataset ($N=300$).

nals driving Auto-Prompt and RAG iteration. We compare it on the En–Zh slice with COMET (Rei et al., 2020) and a single-agent LLM judge. As summarized in Table 4, accuracy rises from 49.7% (COMET) to 76.5% (single-agent LLM judge) to 84.8% (modular multi-agent evaluator): decomposing translation quality into specialized dimensions captures the domain-specific, context-sensitive demands of support interactions that metric-based scoring misses.

5 Online Experiment

We ran a two-week A/B test on continuous live member and customer support traffic. Users were randomized once with fixed 50/50 assignment and remained in one arm for the experiment, preventing repeated-user contamination across conditions. The control was the original production agent—a handcrafted prompt, fixed retrieval pipeline, and pe-

riodic human QA. The treatment was the integrated self-evolved workflow, which bundles (i) evolved Auto-Prompt, (ii) agent-invoked RAG, and (iii) closed-loop evaluator-driven iteration with gated rollout. The three metrics use separate randomized subpopulations.

The experiment was planned for four weeks with sequential monitoring and concluded after two weeks, when the effects were stable, power exceeded 99.9%, and significance remained under alpha-spending corrections. We compare each proportion with a two-sided two-proportion z -test (equivalently, a χ^2 test of independence) and apply Holm correction across the three primary outcomes. The user is the inference unit; average conversations per user are approximately 1.26 for QA and 1.53 for cancellation. The corresponding cluster adjustment is modest (design effect ≤ 1.5), and user-clustered GEE/CR2 standard errors preserve significance for all outcomes; for example, the cancellation statistic changes from $z = 10.0$ to $z = 8.1$. Table 5 reports the unclustered point estimates and confidence intervals.

QA self-serve. This is the share of product-question or technical conversations resolved without human escalation. It increased from 33.7% to 42.7%, an absolute lift of 9.0 percentage points (95% CI [8.4, 9.6]; $z = 27.6$).

Cancellation self-serve. This is the share of cancellation-intent conversations completed end-to-end without handoff. It increased from 61.9% to 66.6%, an absolute lift of 4.8 percentage points (95% CI [3.8, 5.7]; $z = 10.0$).

Routing accuracy. This is the share routed to the correct human queue against the labeled target. It increased from 38.2% to 68.8%, an absolute lift of 30.6 percentage points (95% CI [23.6, 37.6]; $z = 8.2$).

All three outcomes have $p \ll 10^{-4}$ and remain significant after Holm correction. We also monitored escalation rate, thumbs-up/down feedback, latency, customer-satisfaction score, and moderation incidents; none regressed under treatment. The online study reports the end-to-end system effect of the integrated self-evolved workflow, while Tables 1–3 isolate the components offline.

Beyond aggregate gains, these results show that the integrated self-evolved workflow improved the measured outcomes over the two-week deployment window, consistent with calls for holistic,

production-oriented evaluation (Liang et al., 2023; Es et al., 2023; Qiao et al., 2025).

Limitations

Evaluator dependence. The closed loop relies on LLM-as-judge evaluators (GPT-4.1) for fitness signals. Our ablations measure alignment with human labels (Table 3); larger samples and cross-checks with open evaluators would further characterize variance and model-specific judge sensitivity (Zheng et al., 2023; Dubois et al., 2024).

Cost and latency of evolution. Genetic prompt search incurs additional inference cost per generation, and end-to-end evolution cycles take hours to days depending on dataset size and fitness budget. Future work could study tighter adaptation windows for rapidly changing knowledge.

Single-tenant evaluation. The online study covers two weeks on one deployment surface at a single enterprise. Longer observation windows and deployments in other domains would test durability and transfer to different tool inventories and operating environments.

Online component attribution. The online treatment bundles Auto-Prompt, agent-invoked RAG, and evaluator-driven iteration as the integrated self-evolved workflow. Factorial deployments could quantify the marginal contribution and interaction of each component.

Retrieval ceiling. Response quality is bounded by retrieval coverage. When the underlying content lake lacks an authoritative document, the agent’s grounding constraints correctly suppress hallucination but cannot synthesize a correct answer; further gains require improvements to retrieval recall and authoritative-content coverage that are orthogonal to the self-evolution loop.

Limited multilingual stress test. The controlled translation study focuses on English $\leftrightarrow$ Chinese (Section 4.4). Systematic evaluation of low-resource languages and code-mixed input remains an important extension.

Closed-source dependencies. The production system depends on closed-source foundation models (GPT-4o-mini, GPT-4.1) and a proprietary search backend. Reproduction with open-source equivalents (e.g., Llama 3, BGE retrievers) is feasible in principle but has not been verified end-to-end.

Table 5: Online Experiment Results

Metric	Control original production agent	Treatment integrated self-evolved workflow	Absolute Lift (95% CI)	z
QA self-serve ¹	33.7%	42.7%	+9.0 pp [8.4, 9.6]	27.6
Cancellation self-serve ²	61.9%	66.6%	+4.8 pp [3.8, 5.7]	10.0
Routing accuracy ³	38.2%	68.8%	+30.6 pp [23.6, 37.6]	8.2

¹ QA: 42,982 / 45,669 conversations and 35,867 / 34,334 users (control / treatment).

² Cancellation: 20,468 / 20,456 conversations and 13,442 / 13,308 users (control / treatment).

³ Routing: 356 / 356 decisions (control / treatment) on a fixed labeled evaluation set.

Two-sided two-proportion z -tests; all $p \ll 10^{-4}$ and significant after Holm correction.

Acknowledgments

We thank Artem Grigoryan, Tony Huynh, Zhentao Lin, Chris Korbel, Umang Lahoti and Ajay Vishwanathan for their support and collaboration on the agent application and A/B test.

Use of AI assistance. In preparing this manuscript, the authors used Anthropic’s Claude (via Claude Code) for prose polishing, LaTeX formatting and table layout, restructuring of section ordering, and drafting assistance on the Limitations section. All technical claims, experimental design, results, figures, and citations were authored, reviewed, and verified by the human authors, who take full responsibility for the content of the paper.

References

- Akari Asai and 1 others. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *ICLR*.
- Alec Berntson. 2023. [Azure ai search: Outperforming vector search with hybrid retrieval and reranking](#).
- Yann Dubois and 1 others. 2024. Length-controlled alpacaeval: A simple way to debias automatic evaluators. *arXiv:2404.04475*.
- Shahul Es and 1 others. 2023. Ragas: Automated evaluation of retrieval augmented generation. *arXiv:2309.15217*.
- Chrisantha Fernando and 1 others. 2023. Prompt-breeder: Self-referential self-improvement via prompt evolution. *ArXiv:2309.16797*.
- Luyu Gao and 1 others. 2023a. Precise zero-shot dense retrieval without relevance labels. In *ACL*.
- Yunfan Gao and 1 others. 2023b. Retrieval-augmented generation for large language models: A survey. *arXiv:2312.10997*.
- Qingyan Guo and 1 others. 2024. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *ICLR*.
- Kelvin Guu and 1 others. 2020. Realm: Retrieval-augmented language model pre-training. *ICML*.
- John H. Holland. 1992. *Adaptation in Natural and Artificial Systems*. MIT Press.
- Vladimir Karpukhin and 1 others. 2020. Dense passage retrieval for open-domain question answering. In *EMNLP*.
- Omar Khattab and 1 others. 2023. Dspy: Compiling declarative language model calls into self-improving pipelines. *ArXiv:2310.03714*.
- Seungone Kim and 1 others. 2024. Prometheus 2: An open source language model specialized in evaluating other language models. In *EMNLP*.
- Patrick Lewis and 1 others. 2020a. Mlqa: Evaluating cross-lingual extractive question answering. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*.
- Patrick Lewis and 1 others. 2020b. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *NeurIPS*.
- Percy Liang and 1 others. 2023. Holistic evaluation of language models. *Transactions on Machine Learning Research (TMLR)*. *ArXiv:2211.09110*.
- Jimmy Lin and 1 others. 2021. Pretrained transformers for text ranking: Bert and beyond. *Synthesis Lectures on Human Language Technologies*.
- Xi Victoria Lin and 1 others. 2022. Few-shot learning with multilingual generative language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9019–9052.
- Yang Liu and 1 others. 2023. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *EMNLP*.
- Shayne Longpre and 1 others. 2021. [Mkqa: A linguistically diverse benchmark for multilingual open domain question answering](#). *Transactions of the Association for Computational Linguistics*, 9:1389–1406.

- Rodrigo Nogueira and Kyunghyun Cho. 2019. Passage re-ranking with bert. *arXiv:1901.04085*.
- Oded Ovadia and 1 others. 2024. Fine-tuning or retrieval? comparing knowledge injection in llms. *arXiv:2312.05934*.
- Charles Packer and 1 others. 2023. Memgpt: Towards llms as operating systems. *arXiv:2310.08560*.
- Chanhee Park and 1 others. 2025. Mirage: A metric-intensive benchmark for retrieval-augmented generation evaluation. *Findings of NAACL*. ArXiv:2504.17137.
- Ofir Press and 1 others. 2023. Measuring and narrowing the compositionality gap in language models. In *Findings of EMNLP*. ArXiv:2210.03350.
- Shuofei Qiao and 1 others. 2025. Benchmarking agentic workflow generation. In *ICLR*. ArXiv:2410.07869.
- Ricardo Rei and 1 others. 2020. Comet: A neural framework for mt evaluation. *arXiv preprint arXiv:2009.09025*.
- Timo Schick and 1 others. 2023. Toolformer: Language models can teach themselves to use tools. *NeurIPS*.
- Noah Shinn and 1 others. 2023. Reflexion: Language agents with verbal reinforcement learning. *arXiv:2303.11366*.
- Heydar Soudani and 1 others. 2024. Fine tuning vs. retrieval augmented generation for less popular knowledge. *arXiv:2403.01432*.
- Darrell Whitley. 1994. A genetic algorithm tutorial. *Statistics and Computing*.
- Qingyun Wu and 1 others. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv:2308.08155*.
- Shi-Qi Yan and 1 others. 2024. Corrective retrieval augmented generation. *arXiv:2401.15884*.
- Chengrun Yang and 1 others. 2024. Large language models as optimizers. In *ICLR*.
- Shunyu Yao and 1 others. 2023. React: Synergizing reasoning and acting in language models. In *ICLR*.
- Lianmin Zheng and 1 others. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv:2306.05685*.
- Yongchao Zhou and 1 others. 2023. Large language models are human-level prompt engineers. In *ICLR*.